VIT BHOPAL UNIVERSITY
Department of Computer Science & Engineering
Academic Year 2025-26

File Directory Organizer

Submitted By : Mohit Agrawal
Registration No. : 25BCE10672
Course : B.Tech – CSE, 1st Year
Date : 30/03/2026

1. Introduction

Managing files on a computer is a routine yet often neglected task. Over time, folders like Downloads accumulate hundreds of mixed files — photos, PDFs, videos, installers — making retrieval slow and frustrating. This project presents a Python script that automatically organizes a messy folder by moving each file into a named subfolder based on its extension.

The tool uses only Python's built-in os and shutil modules, requires no installation of third-party packages, and runs on Windows, macOS, and Linux.

2. Objectives

- Automatically sort files into category folders (Images, Documents, Media, etc.).
- Handle unknown file types by placing them in an Others folder.
- Prevent data loss by renaming duplicate files instead of overwriting them.
- Provide a clear console log of every action performed.
- Include a test-setup script for safe, risk-free demonstration.

3. System Design

3.1 File Categories

Category	File Extensions
Images	`.jpg .jpeg .png .gif .svg`
Documents	`.pdf .docx .doc .txt .xlsx .pptx .csv`
Archives	`.zip .rar .tar .gz`
Media	`.mp4 .mp3 .mkv .wav`
Executables	`.exe .msi .dmg`
Others	`All unrecognized extensions`

3.2 Program Flow

The script follows a five-step pipeline: (1) receive a target directory path, (2) scan all files using os.listdir(), (3) map each file extension to a category via a dictionary, (4) check for duplicate filenames

at the destination and rename with a _copy suffix if needed, and (5) move the file using shutil.move() and log the result to the console.

4. Implementation

4.1 Technologies Used

- Language: Python 3.x
- os module — directory listing, path handling, folder creation.
- shutil module — high-level file move operations.

4.2 Key Code Snippet

```
for filename in os.listdir(target_dir):
file_path = os.path.join(target_dir, filename)
if os.path.isdir(file_path): continue
_, ext = os.path.splitext(filename)
category = extension_map.get(ext.lower(), 'Others')
dest = os.path.join(target_dir, category, filename)
if os.path.exists(dest):
dest = dest.replace(ext, f'_copy{ext}')
shutil.move(file_path, dest)
```

5. Testing

#	Input File	Expected Folder	Result
1	biology_notes.txt	Documents/	✓ PASS
2	profile_pic.jpg	Images/	✓ PASS
3	podcast_episode.mp3	Media/	✓ PASS
4	game_installer.exe	Executables/	✓ PASS
5	backup_files.zip	Archives/	✓ PASS
6	weird_file.unknown	Others/	✓ PASS
7	Duplicate filename	Renamed _copy	✓ PASS
8	Invalid directory	Error message	✓ PASS

All 8 test cases passed. The organizer correctly handled all file types, duplicate filenames, and invalid paths without errors.

6. Conclusion

The File Directory Organizer demonstrates how a concise Python script can solve a real, everyday problem. Using only two standard library modules it reliably classifies and moves files, handles edge cases gracefully, and provides a clear audit trail. The project reinforced key programming concepts including dictionaries, file I/O, loops, and exception handling.

Future improvements could include recursive subfolder support, an undo log, a Tkinter GUI, and user-defined categories through a JSON config file.

References

[1] Python Software Foundation, *os — Miscellaneous operating system interfaces*, docs.python.org/3/library/os.html

[2] Python Software Foundation, *shutil — High-level file operations*, docs.python.org/3/library/shutil.html

[3] Sweigart, A. (2019). *Automate the Boring Stuff with Python*, 2nd ed. No Starch Press.